SALESTORM

High-Scale E-Commerce Flash Sale System

1.Problem Statement:

SALESTORM is a high-scale e-commerce flash-sale system designed to handle a sudden surge of customers during a limited-stock sale. In this scenario, **10,000 customers may try to purchase a product at the same time while only 100 units are available**. The system must handle these concurrent requests without overselling, prevent duplicate reservations and transactions, and maintain reliable payment and order processing even when services fail.

2.Requirements:

Functional Requirements

The SALESTORM system should:

1. Allow customers to **discover products and manage their cart**.
2. **Check product availability** before purchase.
3. **Reserve inventory safely** during checkout.
4. Release reserved inventory when **payment fails or the reservation expires**.
5. Process payments and prevent **duplicate transactions**.
6. Create and manage orders through their complete lifecycle.
7. Handle **payment, service, and database failures** with proper recovery.
8. Support shipment, delivery tracking, and customer notifications. System design - Hackathon

Non-Functional Requirements

The system should:

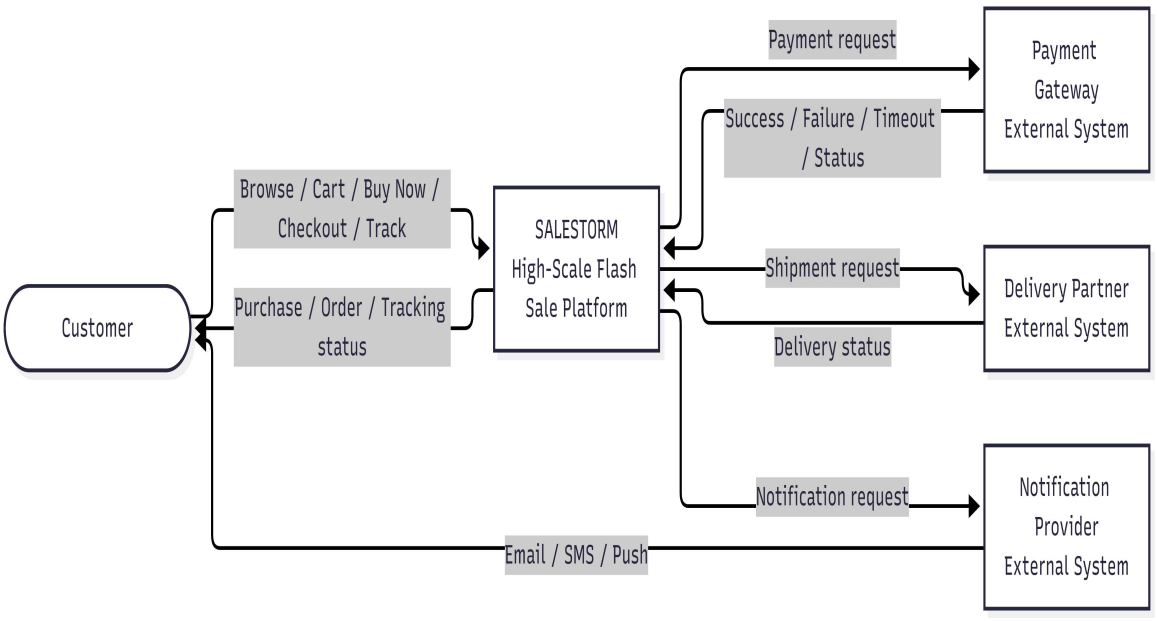
1. Handle **10,000 simultaneous purchase requests**.
2. Ensure **100 available units never result in more than 100 successful sales**.
3. Support **horizontal scaling** during peak traffic.
4. Maintain **inventory consistency and transaction reliability**.

5. Prevent duplicate requests using **idempotency**.
6. Provide defined **failure recovery mechanisms**.
7. Provide **security, monitoring, logging, and tracing** for critical operations.

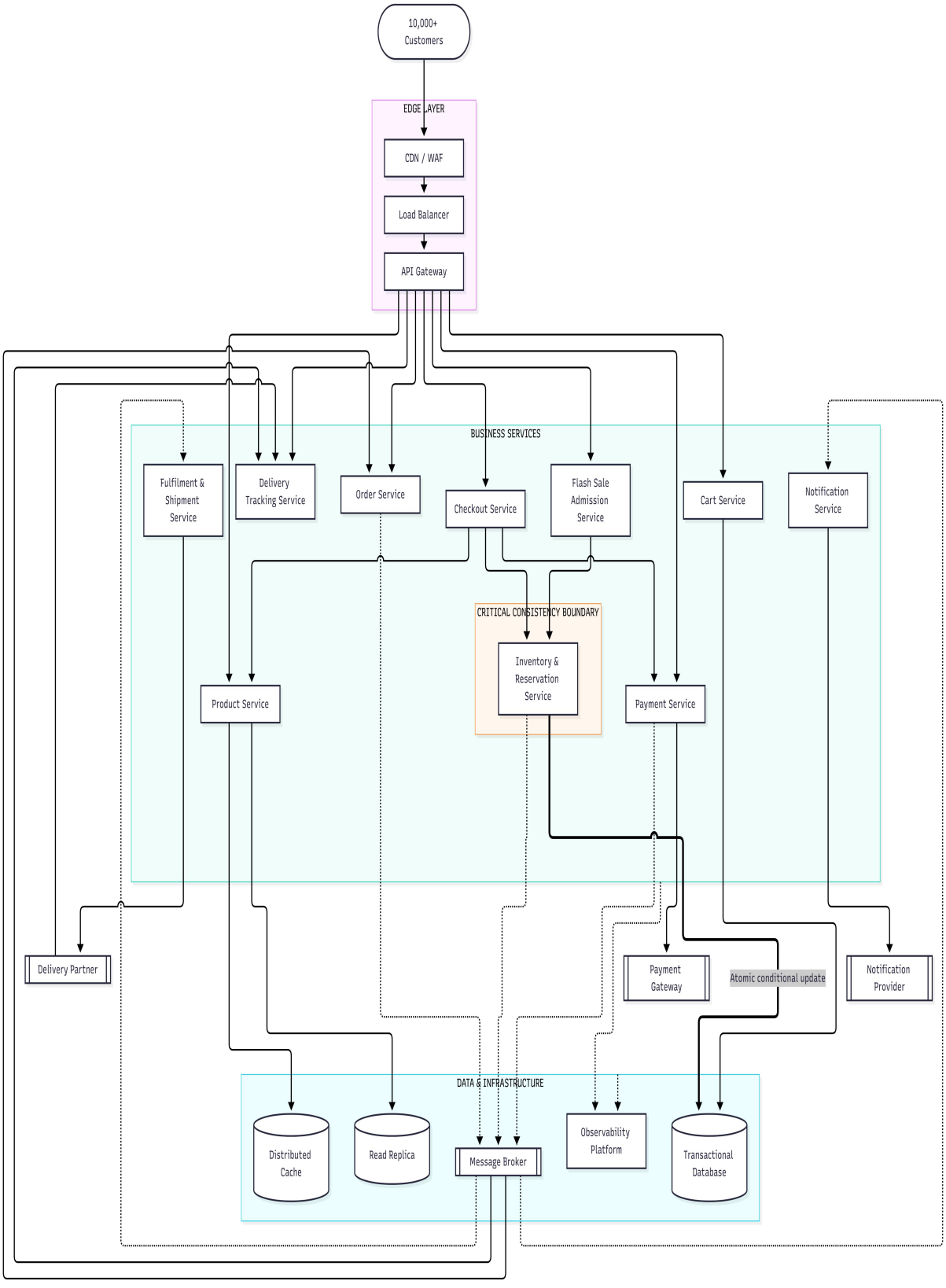
3. Assumptions:

· **Inventory:** Only **100 units** are available for the flash sale.
· **Concurrent Users:** Up to **10,000 customers** may purchase simultaneously.
· **No Overselling:** Successful sales must **never exceed available inventory**.
· **Duplicate Requests:** Customers may send duplicate purchase/payment requests.
· **Service Failures:** Payment or other backend services may temporarily fail.
· **Reservation Expiry:** Unpaid or expired reservations are **released back to inventory**.

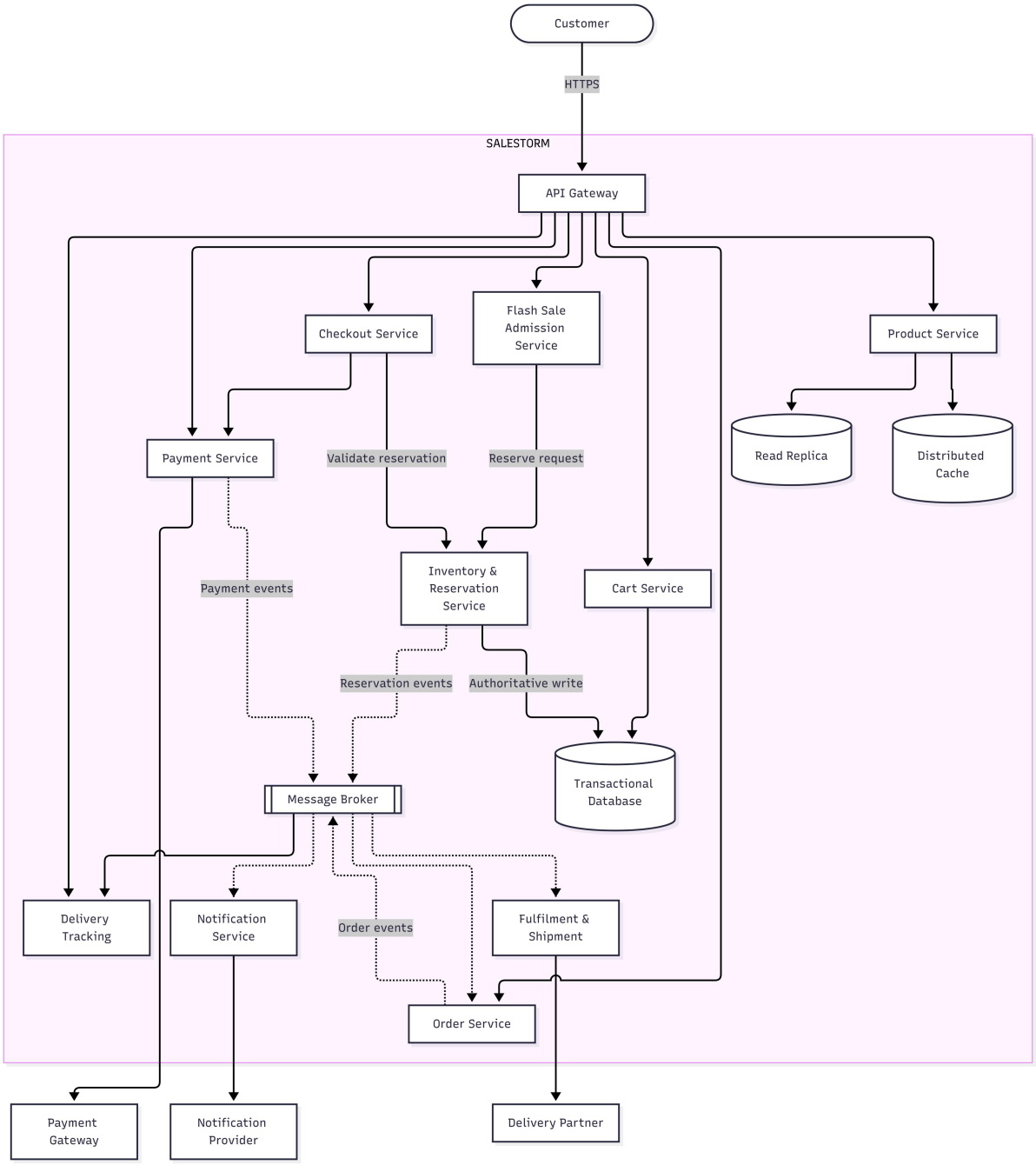
4. System Context Diagram:



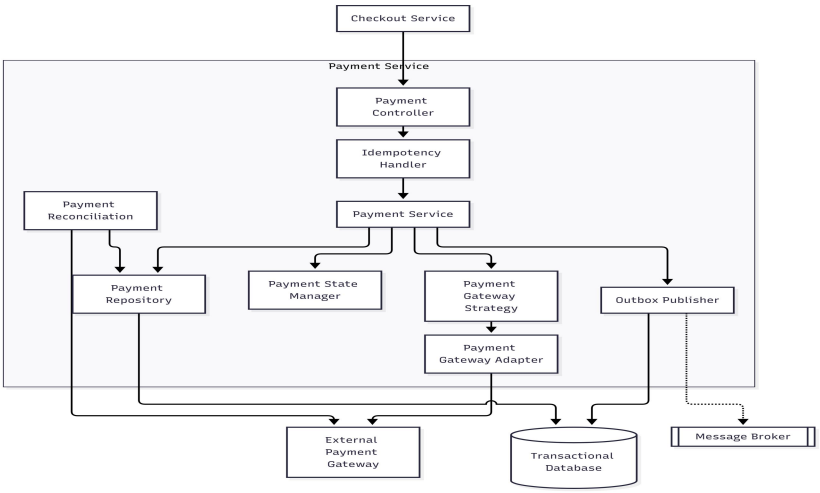
5. HLD Architecture:



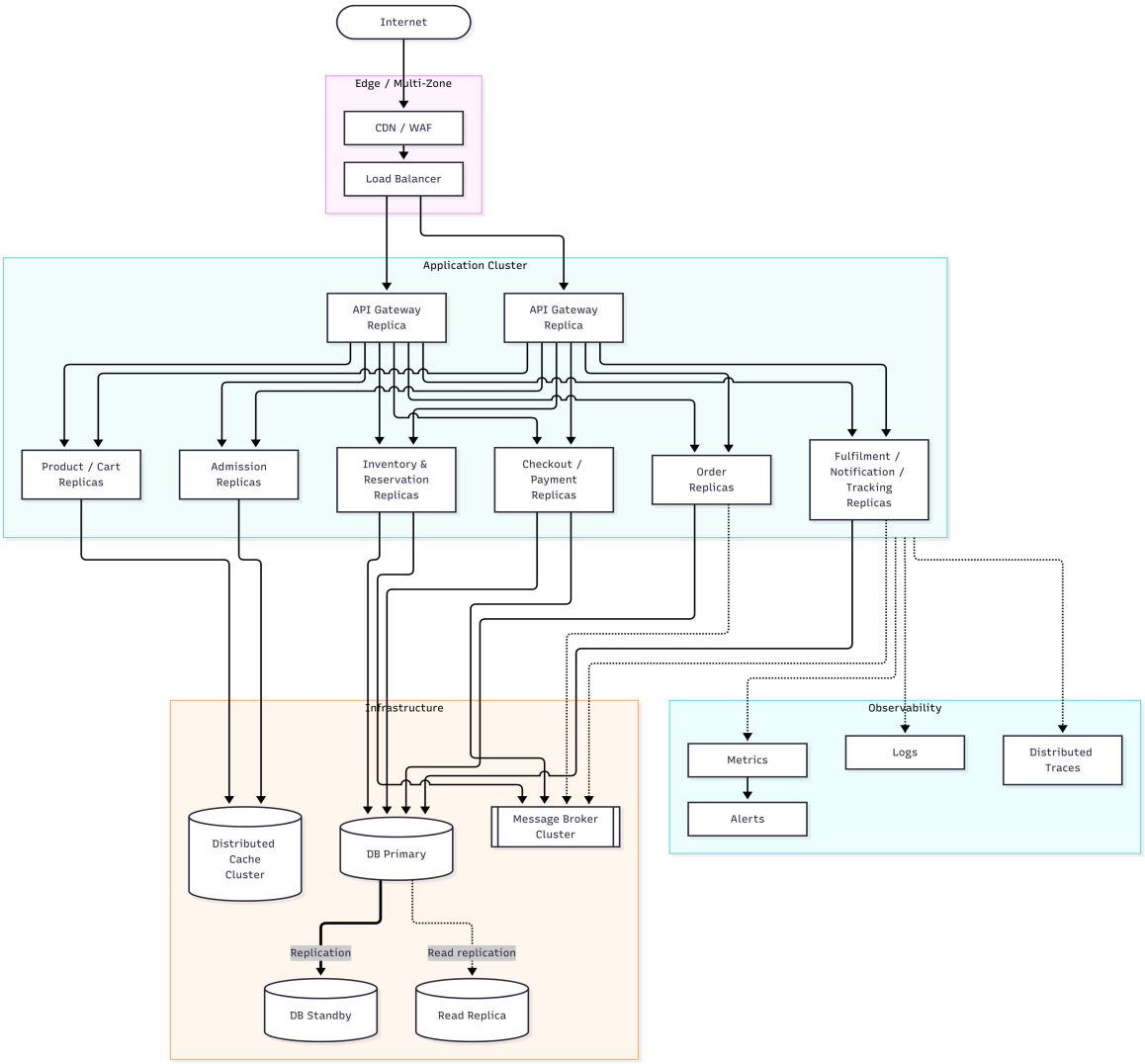
6. Container Diagram:



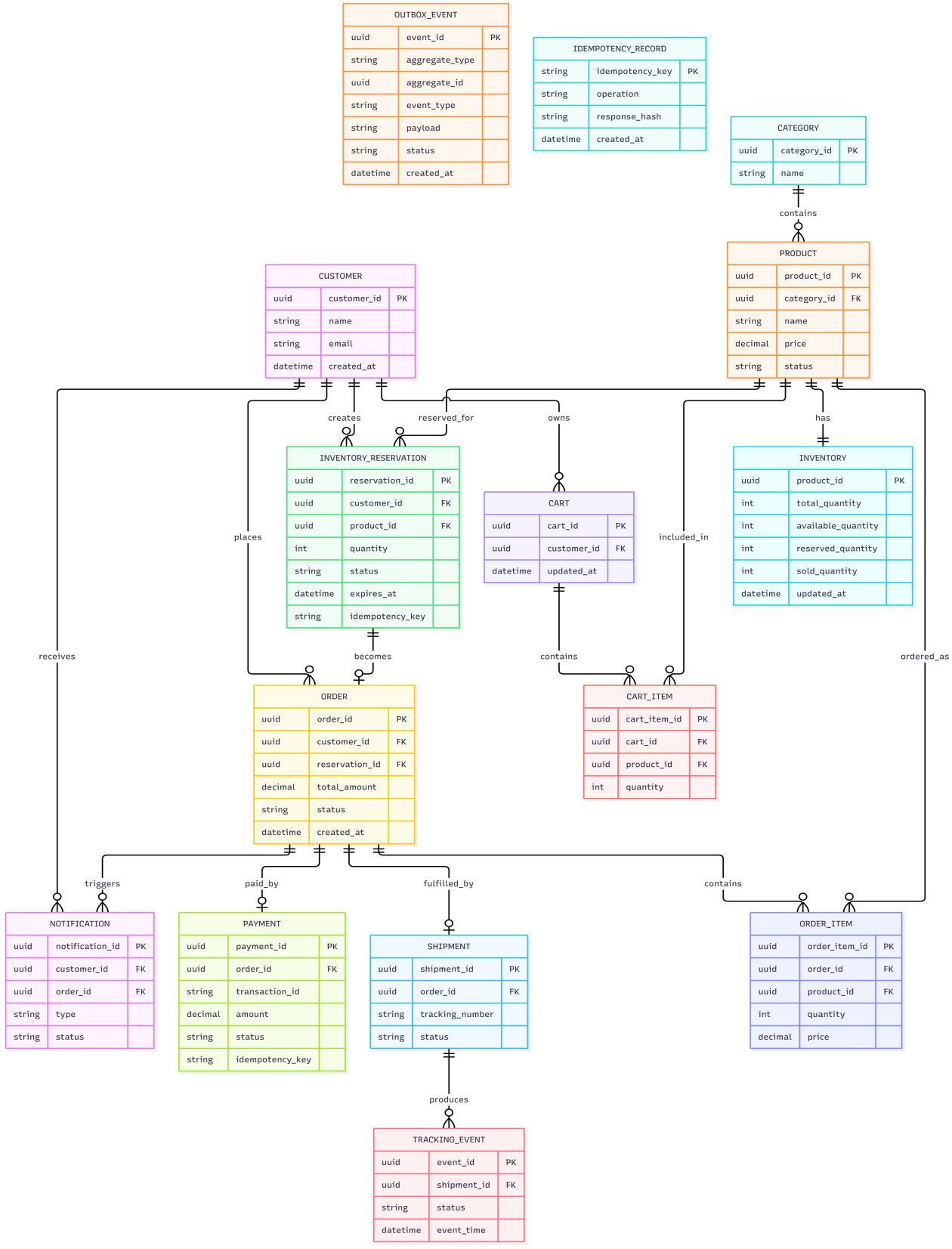
7. Component Diagram:



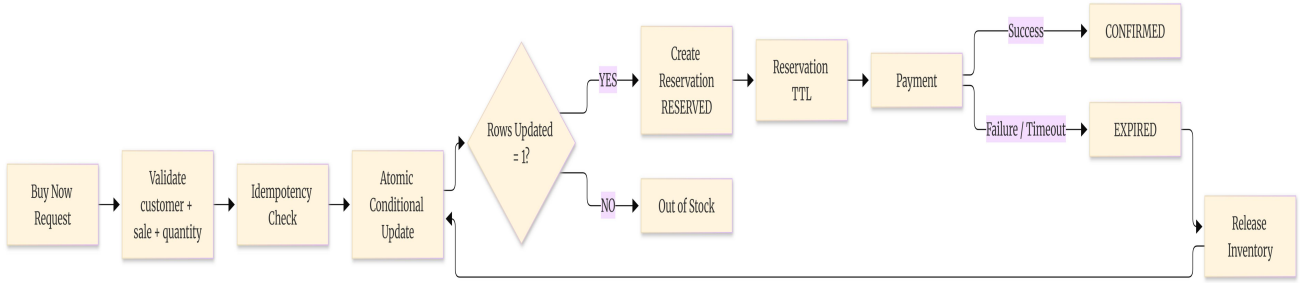
8. Deployment Diagram:



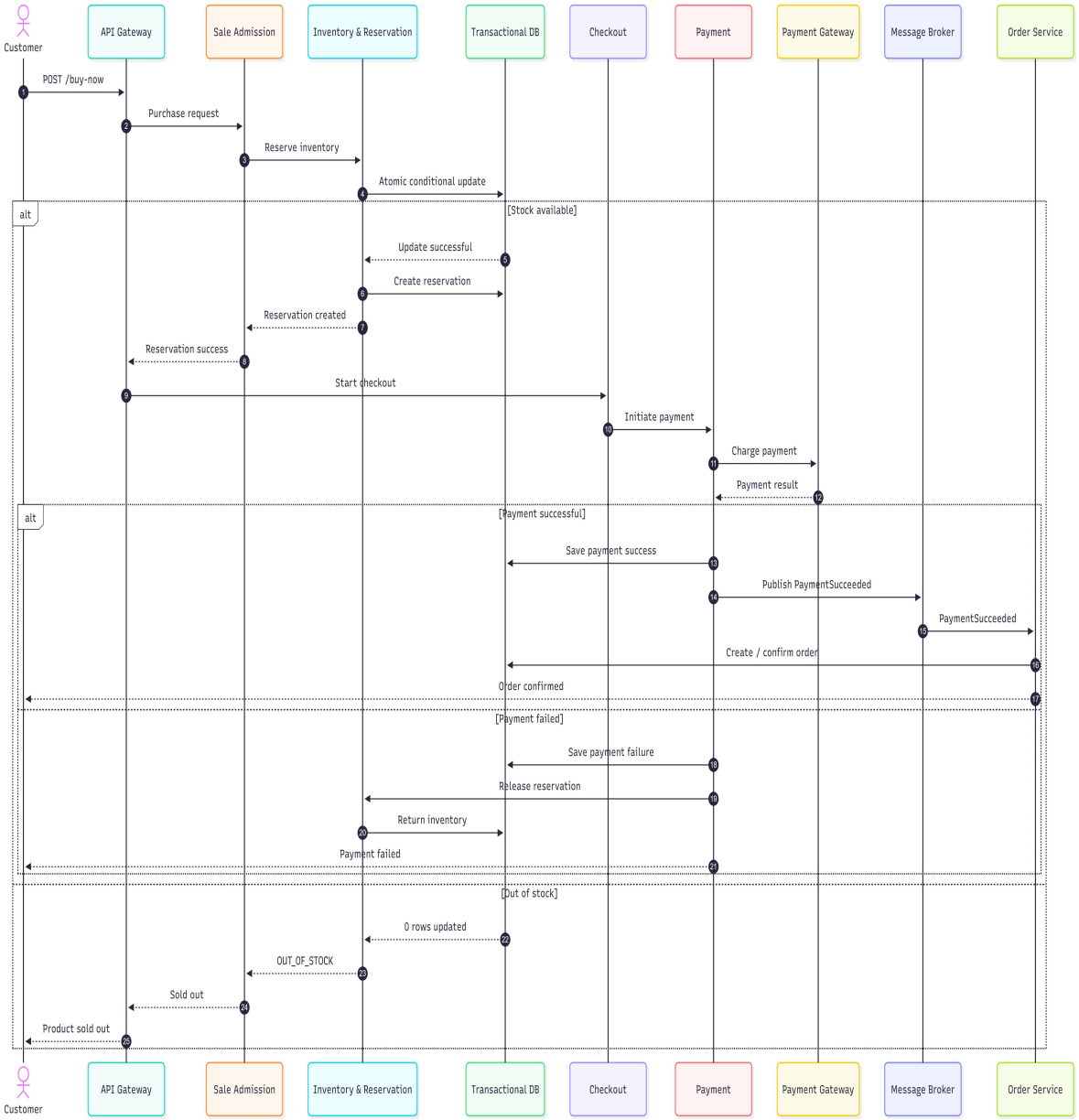
9. Database / ER Diagram:



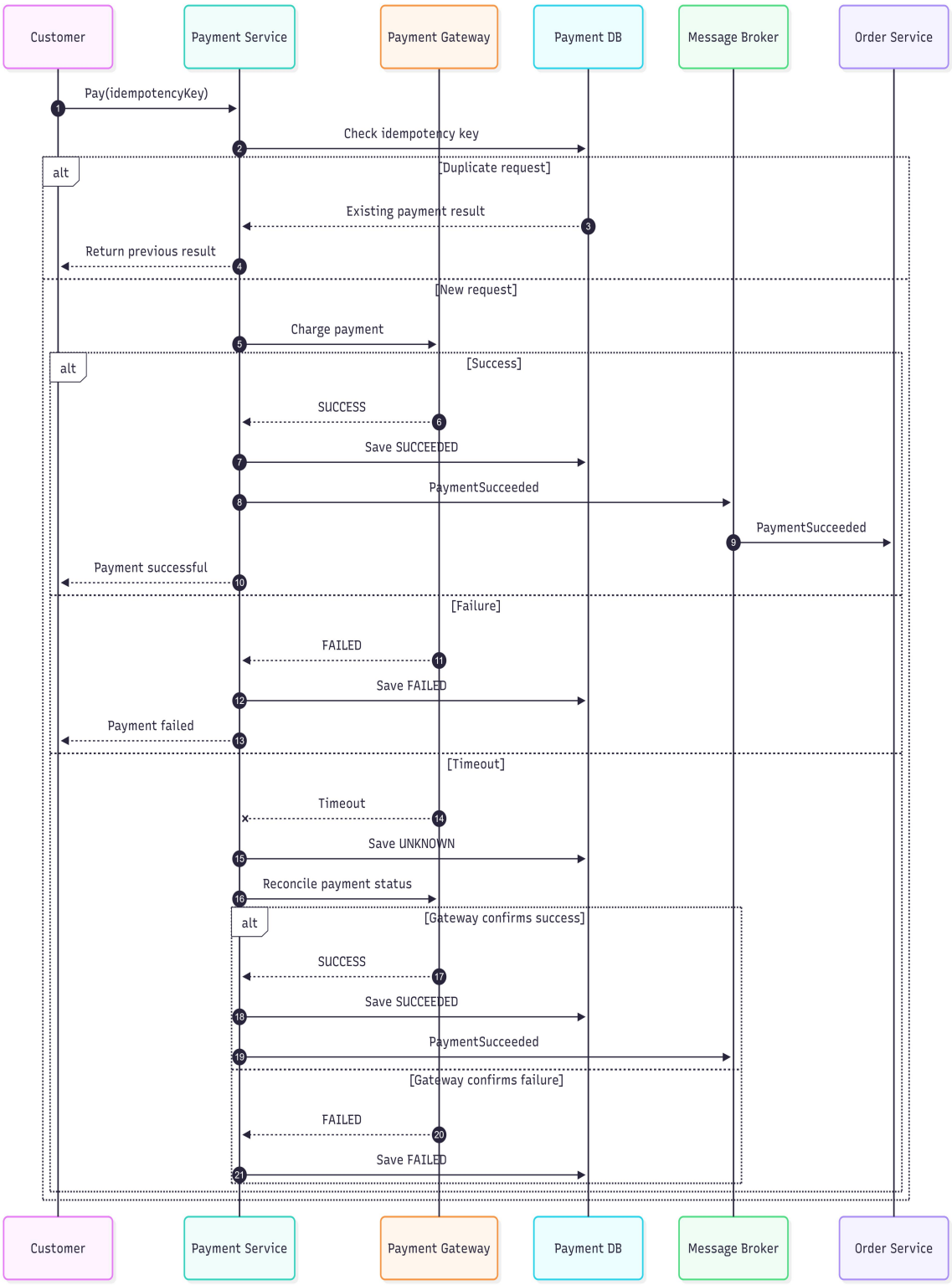
10. Inventory Reservation Design:



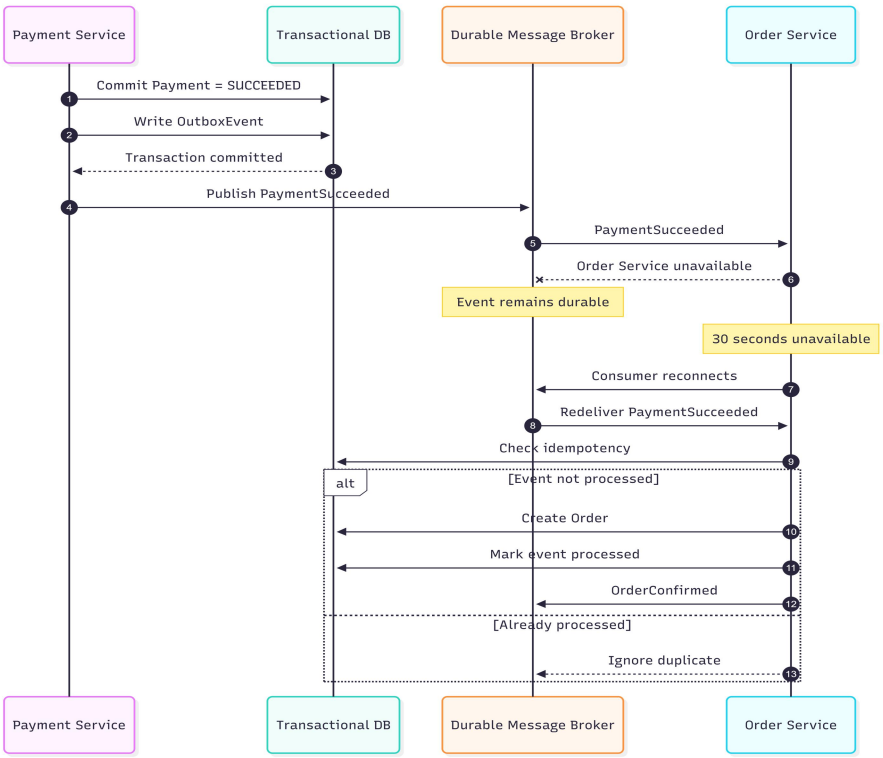
11. Purchase Sequence Diagram:



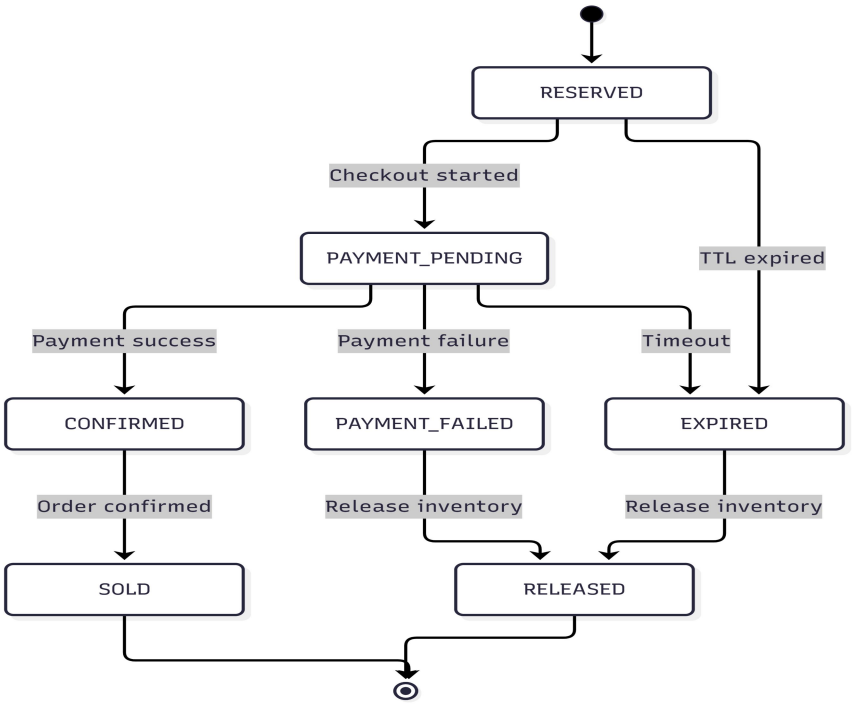
12. Payment Sequence Diagram:



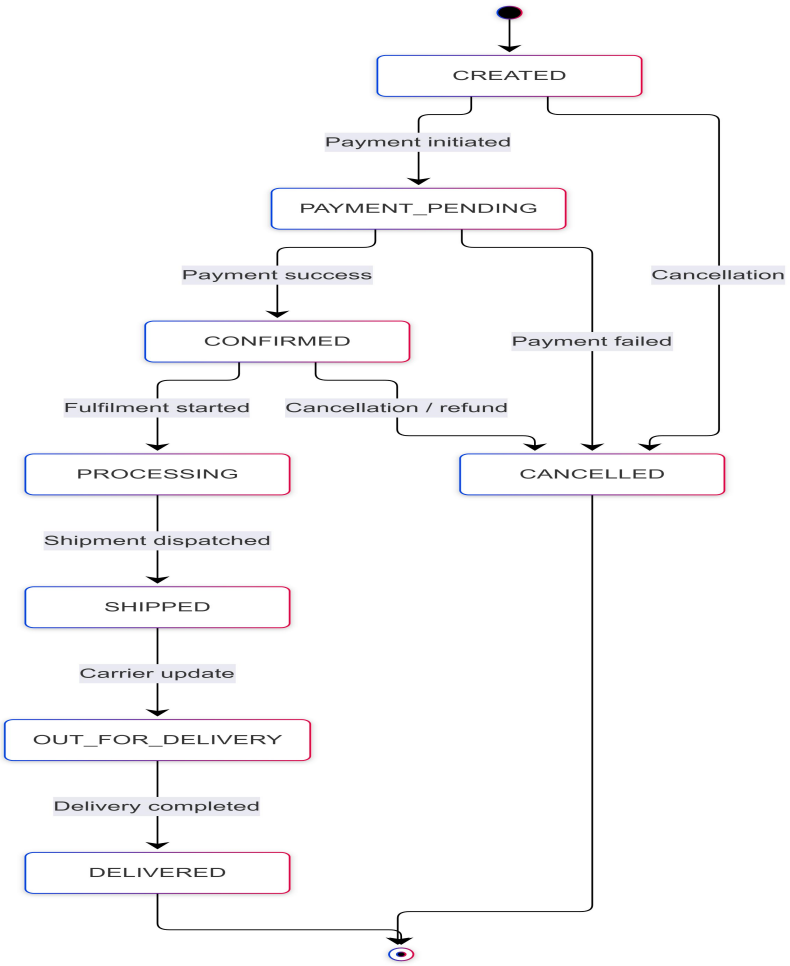
13. Order Sequence Diagram:



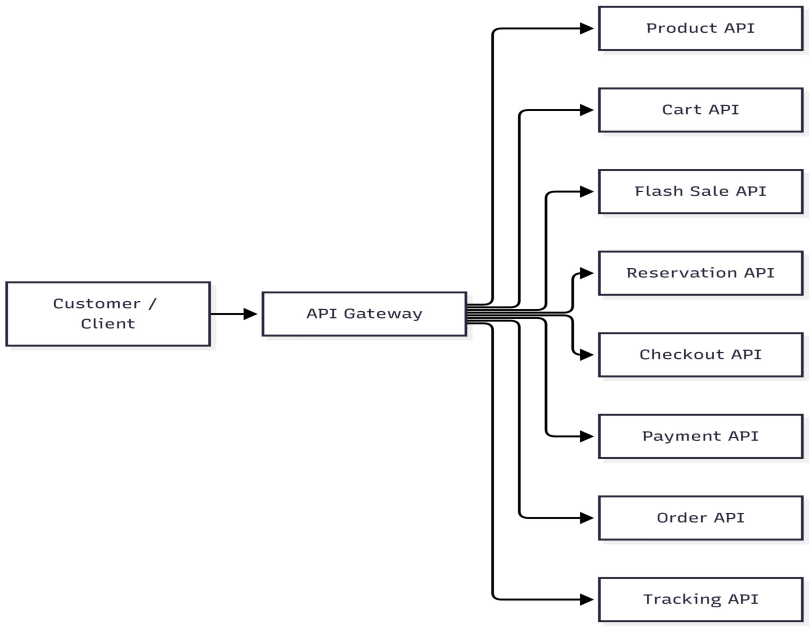
14.Reservation State Diagram:



15.Order State Diagram:

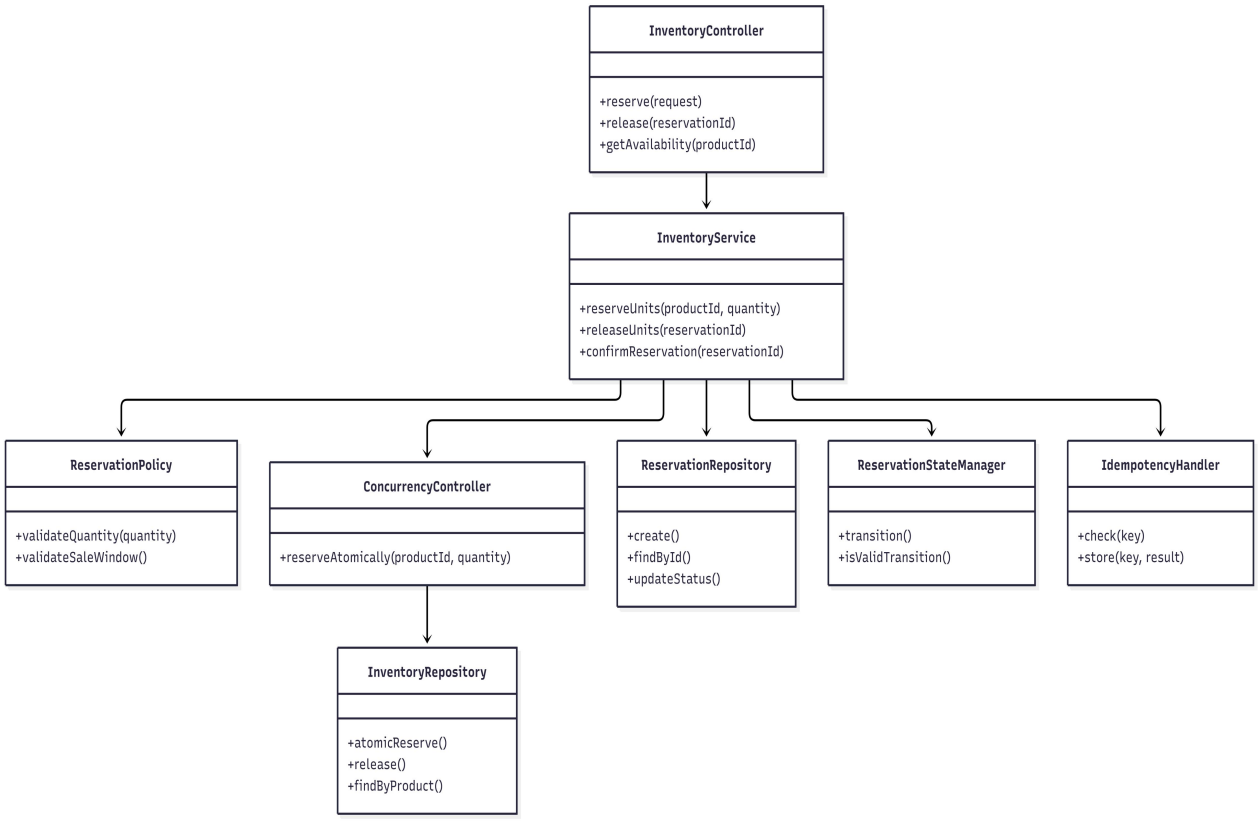


16. API Design:

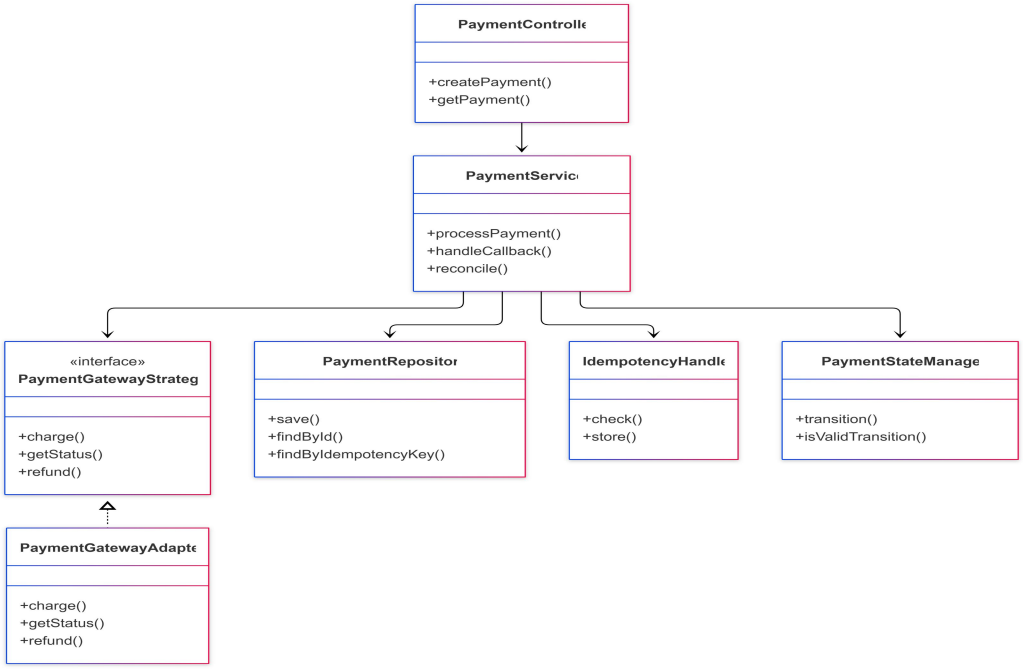


17. Low Level Class Diagram:

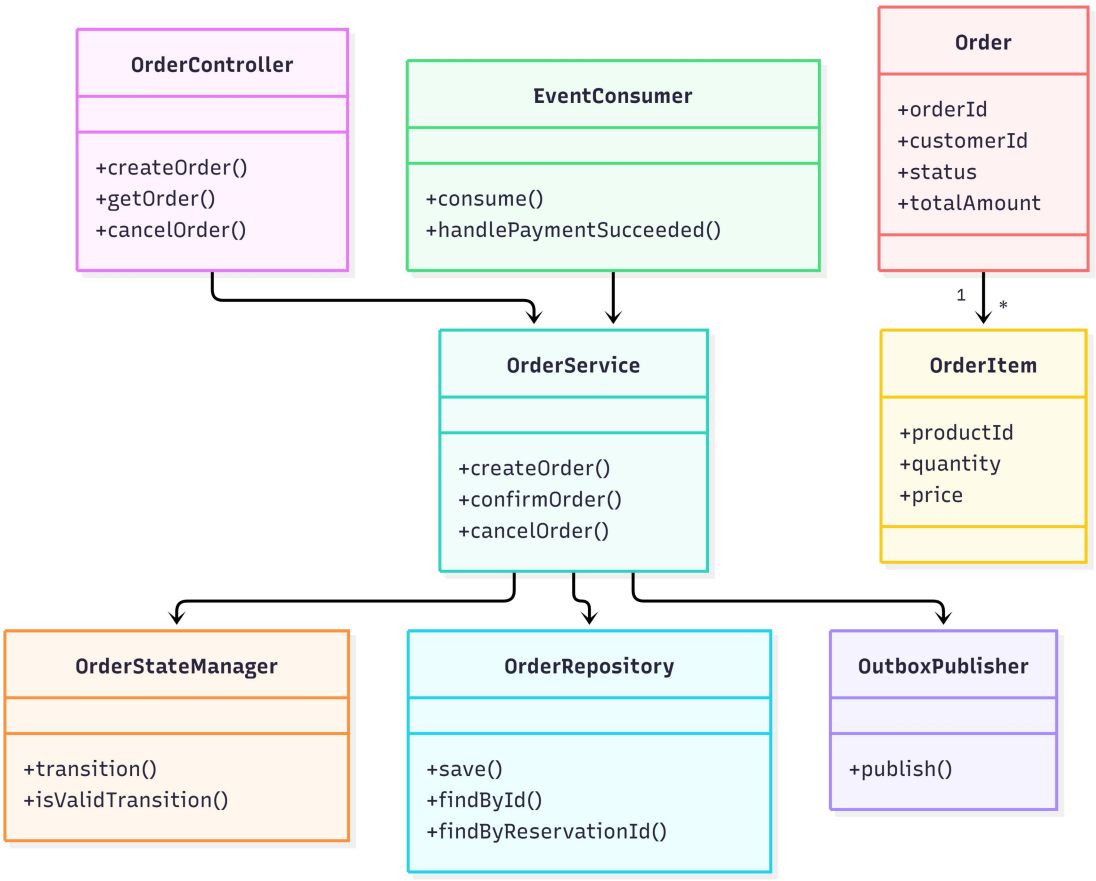
Inventory Class:



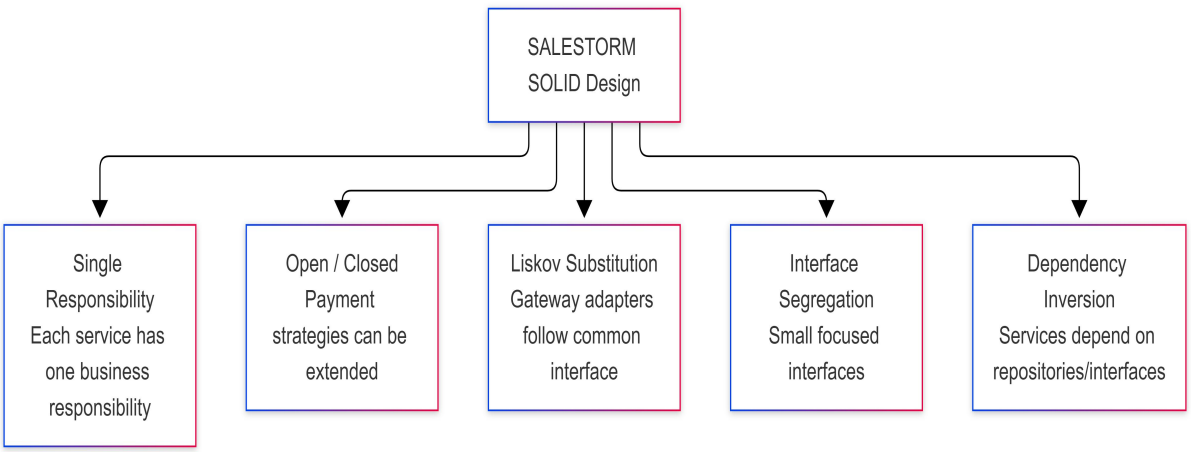
Payment Class:



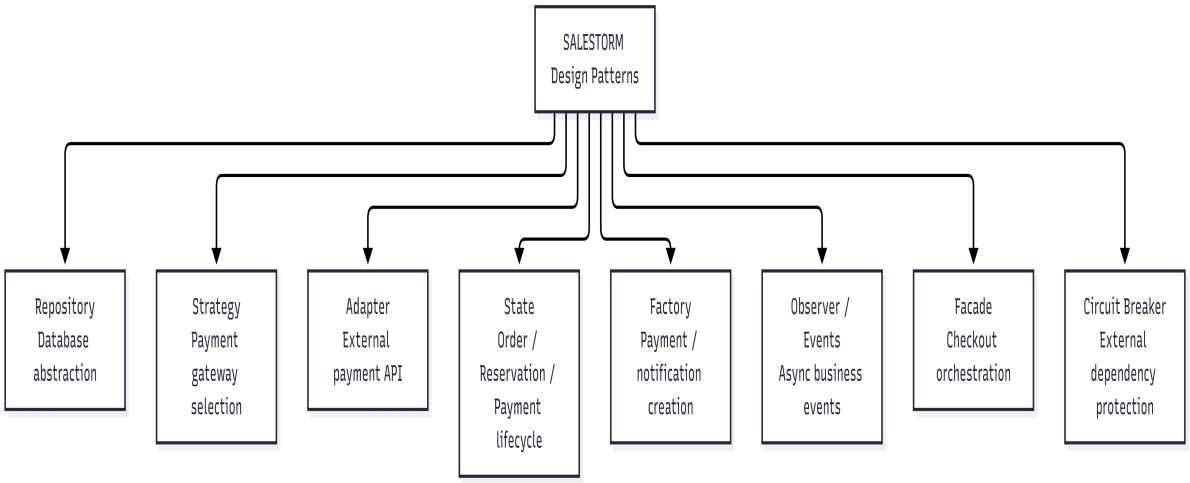
Order Class:



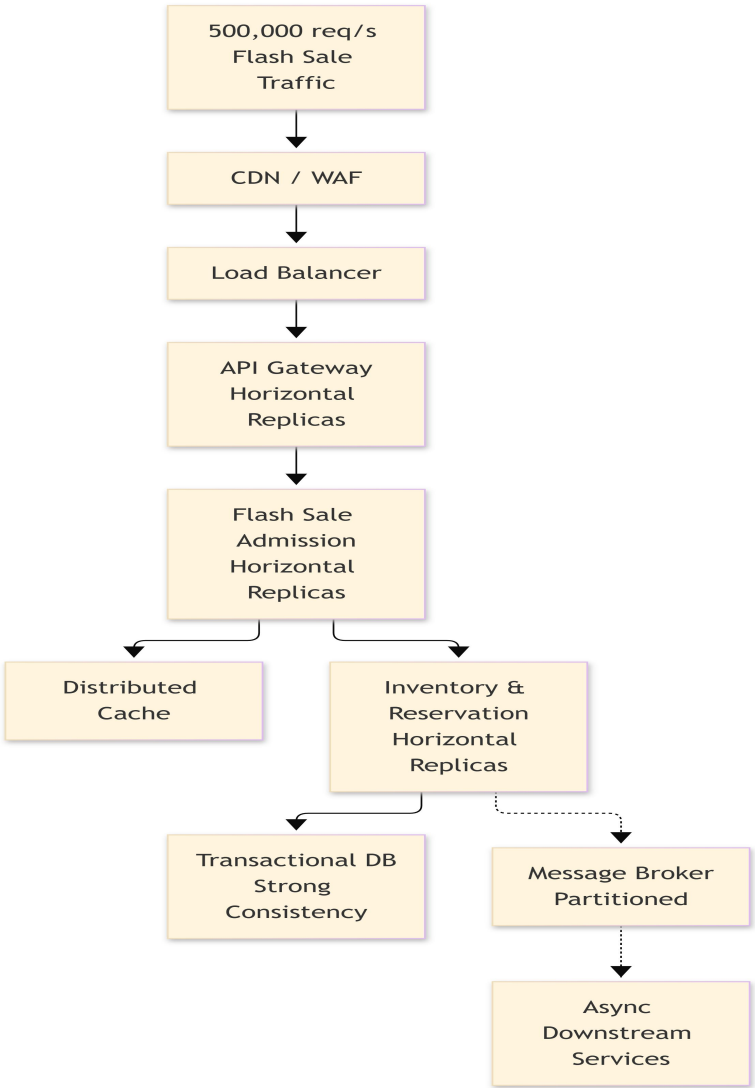
18. SOLID Principle:



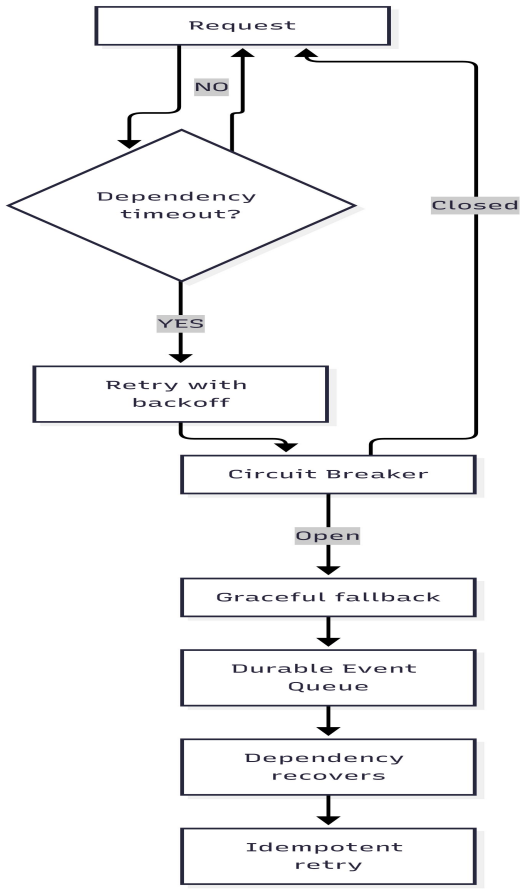
19. Design Pattern:



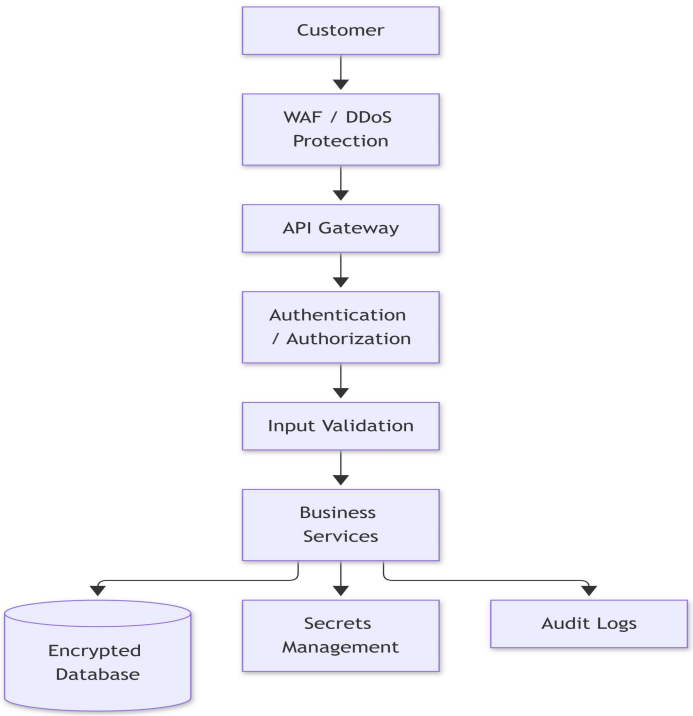
20. Scalability:



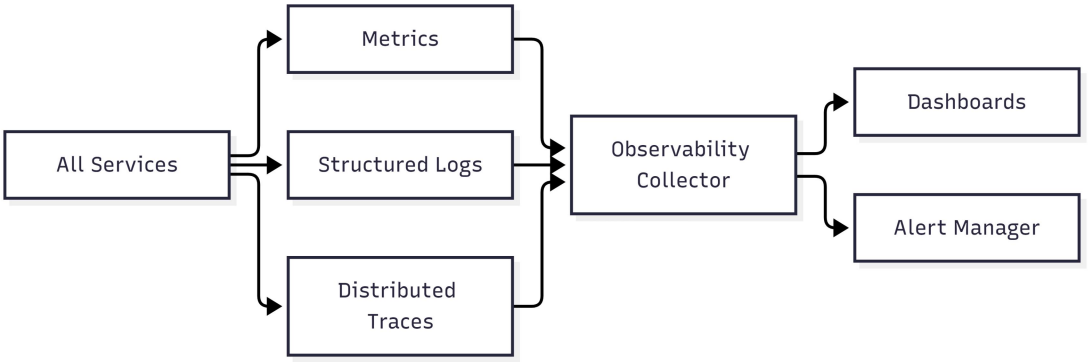
21.Reliability & Failure Handling:



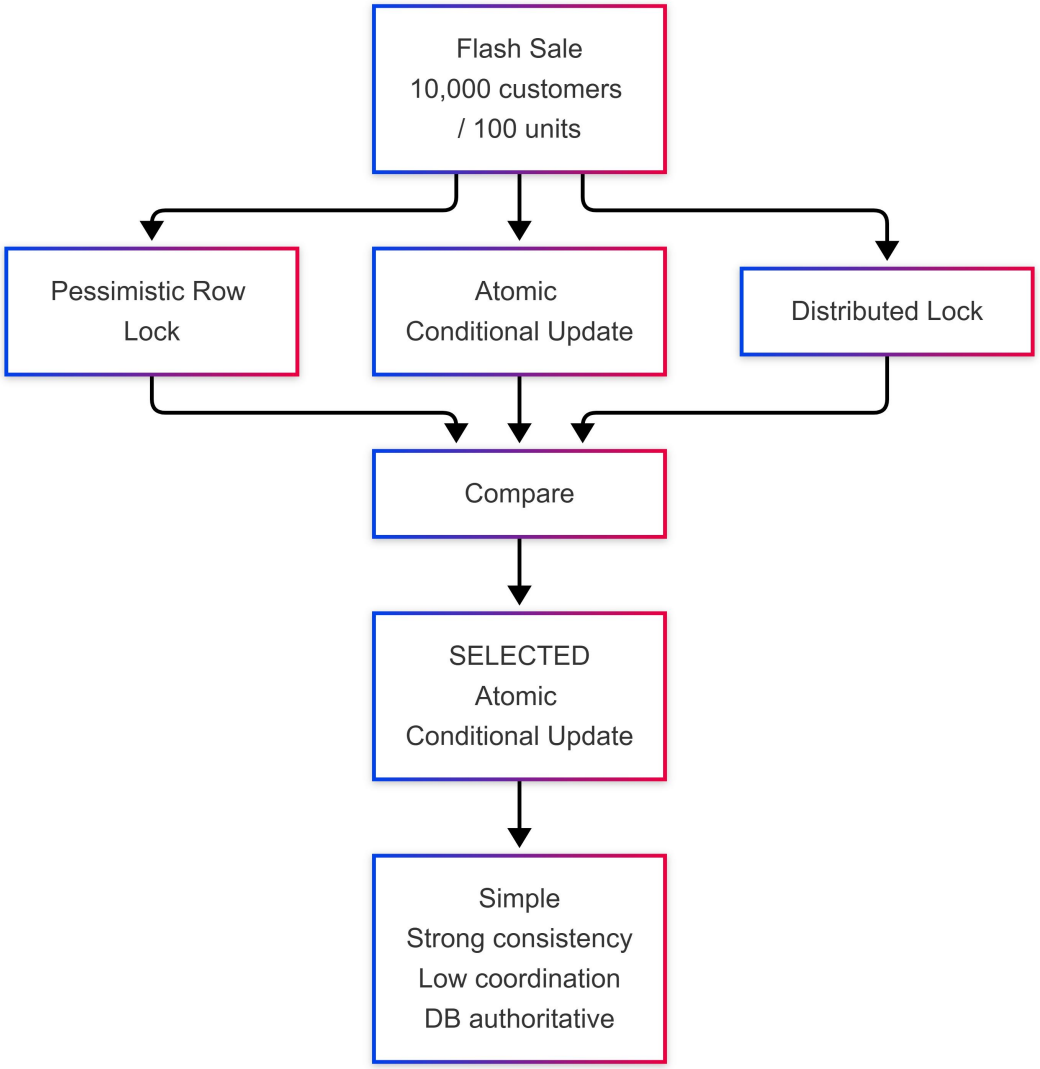
22.Security:



23.Observability:



24.Architecture Decisions:



25.Conclusion:

SALESTORM is designed to provide a scalable, reliable, and consistent flash-sale platform capable of handling extreme traffic spikes while protecting limited inventory. The architecture separates high-volume request handling from the critical inventory decision, with the Inventory & Reservation Service acting as the central consistency boundary.

The design ensures that when 10,000 customers compete for 100 units, inventory is never oversold through an atomic database operation and idempotent reservation processing. Payment and order processing are protected through durable events, retries, and failure recovery, ensuring that successful transactions are not lost even when downstream services temporarily fail.

Overall, SALESTORM focuses on strong inventory consistency, horizontal scalability, fault tolerance, secure APIs, and observable services, while keeping the architecture simple enough to explain, implement, and evolve.